Computational Appendix
Less Left on the Shelf: Reducing Waste at Pasta & Co.

Elisa Regueros Chandler Olson

November 2026

1 Purpose

This appendix records the data-processing rules, implementation, and exact computer output supporting the preliminary results in the main report. The analysis uses an empirical 80% service-level rule and Monte Carlo resampling. It does not use a Poisson model or maximum-likelihood estimation.

2 Data Construction and Checks

The study period is September 13 through October 13, 2026, giving 31 calendar days and 93 bread-day observations. The manager's daily quantities are 3 Rustic Baguettes, 8 Demi Baguettes, and 1 Como Loaf. For each bread b and day t, estimated sales are constructed as

$$\widehat{D}_{b,t} = Q_b^{\text{current}} - W_{b,t},$$

where $W_{b,t}$ is recorded waste. A date absent from the transcribed waste log is treated as having zero waste for all three products. One suspected duplicate set of September 30 entries is excluded.

The resulting dataset has no missing fields, duplicate bread-date keys, or violations of the identity

$$\text{current order} - \text{waste} = \text{estimated sales}.$$

Table 1 summarizes the observations used by the program.

Table 1: Dataset audit by bread type.

Bread	Days	Ordered	Estimated sales	Waste	Zero-waste days
Rustic	31	93	61	32	16
Demi	31	248	213	35	19
Como	31	31	20	11	20
Total	93	372	294	78	55

Zero waste is used as a sellout indicator in the sensitivity analysis. This is an assumption: the available records do not show sellout time or attempted purchases after inventory reached zero. In addition, treating an unlisted date as zero waste should be confirmed with the store's recordkeeping process.

3 Optimization Rule

For observations $D_1, \ldots, D_n$ in a group, the program sorts the values and returns observation

$$D_{(\lceil 0.80n \rceil)}.$$

This is the smallest integer order quantity whose empirical service level is at least 80%. The fixed policy applies this rule to all 31 observations for each bread. The weekday policy applies it separately to groups of four or five observations for each bread-weekday pair.

For a proposed quantity Q and assumed demand D, the program evaluates

$$\text{waste} = \max(Q - D, 0), \qquad \text{unmet demand} = \max(D - Q, 0).$$

Waste cost is waste multiplied by the corresponding wholesale unit cost. Service level is the percentage of the 93 bread-day observations for which $D \leq Q$.

4 Baseline Results

The baseline sets assumed demand equal to estimated sales, including on zero-waste days. The fixed optimized quantities are therefore identical to the current quantities: Como 1, Demi 8, and Rustic 3. The weekday result is shown in Table 2.

Table 2: Baseline weekday optimized quantities.

Weekday	Como	Demi	Rustic
Monday	1	8	3
Tuesday	1	8	3
Wednesday	1	8	2
Thursday	1	8	3
Friday	1	8	3
Saturday	1	8	3
Sunday	1	8	3

Table 3: Baseline in-sample strategy comparison.

Strategy	Wasted units	Waste cost	Service (%)	Unmet units
Current	78	\$151.58	100.00	0
Fixed optimized	78	\$151.58	100.00	0
Weekday optimized	74	\$141.62	100.00	0

The weekday policy reduces in-sample waste by 4 units, or $4/78 = 5.13\%$, and reduces wholesale waste cost by \$9.96. All four units are Rustic Baguettes on Wednesdays. These are in-sample calculations, not out-of-sample performance estimates.

The 100% baseline service level should not be interpreted as 100% of actual customer demand being met. Estimated sales cannot exceed the historical order quantity, so current orders cover every baseline observation by construction.

5 Censored-Demand Sensitivity Analysis

For a sensitivity value $k \in \{0, 1, 2\}$, the program defines

$$D_{b,t}^{(k)} = \widehat{D}_{b,t} + k\,\mathbf{1}\{W_{b,t} = 0\}.$$

The values $k = 1$ and $k = 2$ are hypothetical scenarios, not observed demand. Table 4 gives the exact program output.

Table 4: Sensitivity results by assumed additional sellout demand.

k	Strategy	Waste	Waste cost	Service (%)	Unmet units
0	Current	78	\$151.58	100.00	0
0	Fixed optimized	78	\$151.58	100.00	0
0	Weekday optimized	74	\$141.62	100.00	0
1	Current	78	\$151.58	40.86	55
1	Fixed optimized	116	\$238.52	100.00	0
1	Weekday optimized	108	\$218.60	100.00	0
2	Current	78	\$151.58	40.86	110
2	Fixed optimized	154	\$325.46	100.00	0
2	Weekday optimized	142	\$295.58	100.00	0

The fixed quantities become $(\text{Como}, \text{Demi}, \text{Rustic}) = (2, 9, 4)$ when $k = 1$ and $(3, 10, 5)$ when $k = 2$. The weekday quantities generally rise by the same increment. Wednesday Rustic remains 2 because none of its four observations is classified as a sellout in the transcribed data.

The current policy's waste remains 78 in these scenarios because additional demand is added only where recorded waste was already zero. Its modeled service falls to 40.86% because 55 of 93 bread-day observations are treated as having demand above the current quantity. These sensitivity results show that conclusions depend strongly on the censored-demand assumption.

6 Monte Carlo Validation

The program performs 10,000 simulations using random seed 381. For each of the 93 bread-day positions in the study period, it samples demand with replacement from the historical observations having the same bread type and weekday. All three strategies are evaluated on the same sampled demand in each run, allowing paired comparisons. Table 5 reports the mean results and the 2.5th and 97.5th percentiles of total waste.

Table 5: Monte Carlo results from 10,000 empirical-resampling runs.

Strategy	Mean waste	95% low	95% high	Mean cost	Service	Unmet
Current	78.27	57	100	\$151.96	100%	0
Fixed optimized	78.27	57	100	\$151.96	100%	0
Weekday optimized	74.27	53	96	\$142.00	100%	0

In paired comparisons, fixed optimization reduces waste by 0 units in every run. Weekday optimization reduces waste by exactly 4 units in every run, so its simulated probability of producing

less waste than the current policy is 1.00. This result is constant because the policies differ only for the four Wednesday Rustic positions, and every demand value in that empirical pool is at most 2.

This simulation verifies the calculations under the fitted empirical model; it is not independent evidence that the policy will save exactly four units in a future month. Resampling the same four or five weekday observations cannot represent demand values or external conditions absent from the data.

7 Exact Console Output

The following output was produced by running `python pasta_bread_cleaned_model.py` with Python 3.14, pandas 3.1.1, and NumPy 2.5.4.

```
FIXED OPTIMAL QUANTITIES:
{'Como': 1, 'Demi': 8, 'Rustic': 3}

WEEKDAY OPTIMAL QUANTITIES:
          Como Demi Rustic
Monday 1 8 3
Tuesday 1 8 3
Wednesday 1 8 2
Thursday 1 8 3
Friday 1 8 3
Saturday 1 8 3
Sunday 1 8 3

STRATEGY COMPARISON:
                  wasted_units waste_cost service_level_pct unmet_units
Current 78.0 151.58 100.0 0.0
Fixed optimized 78.0 151.58 100.0 0.0
Weekday optimized 74.0 141.62 100.0 0.0

ADDITIONAL DEMAND ON SELLOUTS: 0
                  wasted_units waste_cost service_level_pct unmet_units
Current 78.0 151.58 100.0 0.0
Fixed optimized 78.0 151.58 100.0 0.0
Weekday optimized 74.0 141.62 100.0 0.0

ADDITIONAL DEMAND ON SELLOUTS: 1
                  wasted_units waste_cost service_level_pct unmet_units
Current 78.0 151.58 40.86 55.0
Fixed optimized 116.0 238.52 100.00 0.0
Weekday optimized 108.0 218.60 100.00 0.0

ADDITIONAL DEMAND ON SELLOUTS: 2
                  wasted_units waste_cost service_level_pct unmet_units
Current 78.0 151.58 40.86 110.0
Fixed optimized 154.0 325.46 100.00 0.0
Weekday optimized 142.0 295.58 100.00 0.0

MONTE CARLO RESULTS (10,000 RUNS):
                  mean_waste waste_95_low waste_95_high mean_waste_cost mean_service_pct mean_unmet
strategy
Current 78.27 57.0 100.0 151.96 100.0 0.0
Fixed optimized 78.27 57.0 100.0 151.96 100.0 0.0
Weekday optimized 74.27 53.0 96.0 142.00 100.0 0.0

PAIRED WASTE COMPARISONS:
                  mean_waste_reduction reduction_95_low reduction_95_high probability_less_waste
strategy
Fixed optimized 0.0 0.0 0.0 0.0
Weekday optimized 4.0 4.0 4.0 1.0
```

8 Reproducibility and Limitations

The baseline optimization and evaluation use the same observations. The reported 5.13% reduction is therefore a historical fit and is not a validated forecast. Each weekday group contains only four or five observations. With four observations, the empirical 80th percentile is the maximum observation; with five, it is the fourth ordered observation. Weekday recommendations can therefore change substantially when a single observation changes.

Additional waste records would improve sample size but would not by themselves identify demand above the current stock level. Stockout times, attempted purchases after sellout, or observations collected under higher order quantities would provide information about censored demand.

The Monte Carlo analysis uses the empirical weekday distributions, 10,000 runs, and random seed 381. It quantifies simulation variability under those distributions but does not remove the underlying censoring or small-sample limitations. The sensitivity analysis separately changes all zero-waste observations by the same deterministic amount.

9 Complete Python Source

The complete analysis program is included directly from the repository below. Keeping the listing linked to the source file prevents the appendix from silently diverging from the executed code.

```
"""

Important assumptions:

- Daily historical deliveries are 3 Rustic, 8 Demi, 1 Como.
- A date not appearing in the waste log had zero waste for these breads.
- One suspected duplicate set of September 30 waste entries is excluded.
- Observed sales are used as demand in the baseline run, even on sell-out days.

"""

import pandas as pd
import numpy as np

# ---------------------------------------------------
# 1. HISTORICAL DATA AND CURRENT ORDER QUANTITIES
# ---------------------------------------------------

start = "2026-09-13"
end = "2026-10-13"

# Daily order quantities confirmed by the manager
Q0 = {
    "Rustic": 3,
    "Demi": 8,
    "Como": 1
}

# Wholesale cost per unit (used only to report waste cost)
COST = {
    "Rustic": 2.49,
    "Demi": 0.97,
    "Como": 3.45
}
# This is a dict of observed waste counts for each day in the historical period.
```

```
# Each tuple is (Rustic, Demi, Como).
#
# Missing dates are assumed to have zero bread waste.
# The possible duplicate on September 30 is excluded.


observed_waste = {
    "2026-09-13": (2, 0, 1),
    "2026-09-14": (0, 2, 1),
    "2026-09-15": (2, 0, 1),
    "2026-09-16": (2, 0, 0),
    "2026-09-17": (1, 0, 0),
    "2026-09-18": (0, 1, 1),
    "2026-09-19": (2, 0, 0),
    "2026-09-20": (3, 0, 0),
    "2026-09-23": (1, 6, 0),
    "2026-09-24": (0, 0, 1),
    "2026-09-27": (0, 0, 1),
    "2026-09-28": (2, 1, 0),
    "2026-09-29": (3, 5, 1),
    "2026-09-30": (2, 2, 0),
    "2026-10-01": (2, 5, 1),
    "2026-10-06": (0, 1, 0),
    "2026-10-07": (2, 3, 1),
    "2026-10-08": (0, 2, 0),
    "2026-10-09": (2, 0, 0),
    "2026-10-11": (0, 5, 0),
    "2026-10-12": (3, 2, 1),
    "2026-10-13": (3, 0, 1),
}

# ------------------------------------------------
# 2. CREATE THE DAILY DATASET
# ------------------------------------------------

# These are merely houskeeping steps to create a dataframe with the following columns:
# - date: the date of the observation
# - weekday: the day of the week
# - bread: the type of bread (Rustic, Demi, Como)
# - current_order: the current order quantity for that bread
# - waste: the observed waste for that bread on that date
# - estimated_sales: the estimated sales for that bread on that date (current order minus waste)
# - wholesale_cost: the wholesale cost per unit for that bread
rows = []

for date in pd.date_range(start, end):

    counts = observed_waste.get(
        str(date.date()), (0, 0, 0)
    )

    for bread, wasted in zip(Q0, counts):

        rows.append({
            "date": date,
            "weekday": date.day_name(),
            "bread": bread,
            "current_order": Q0[bread],
            "waste": wasted,
```

```
            "estimated_sales": Q0[bread] - wasted,
            "wholesale_cost": COST[bread]
        })

df = pd.DataFrame(rows)

# ------------------------------------------------
# 3. FIND THE MINIMUM ORDER MEETING 80% OF DEMAND
# ------------------------------------------------

# This function returns the smallest integer quantity that covers at least the specified
# service amount, which is currently 80%. It takes the parameters of the observed
# sales and the service level and returns the minimum order quantity that
# meets the service level requirement.
def minimum_order(values, service=0.80):

    values = sorted(values)

    # Position of the 80th percentile
    position = int(np.ceil(service * len(values))) - 1

    return int(values[position])

# ------------------------------------------------
# 4. OPTIMIZE THE THREE ORDERING STRATEGIES
# ------------------------------------------------

# this function calculates the optimal order quantities for each of the three strategies:
# current orders, fixed optimization, and day-of-week optimization. It returns the fixed
# optimal quantities, weekday optimal quantities, and a comparison of the results for
# each strategy.
def make_policies(sellout_extra=0, service=0.80):
    data = df.copy()

    # Baseline assumption: demand equals observed sales.
    # Sensitivity analysis: add 1 or 2 units on sold-out days.
    data["assumed_demand"] = (
        data["estimated_sales"]
        + np.where(data["waste"] == 0, sellout_extra, 0)
    )

    # STRATEGY 2: FIXED OPTIMIZATION
    # One optimized quantity per bread type, every day.
    fixed = (
        data.groupby("bread")["assumed_demand"]
        .apply(lambda x: minimum_order(x, service))
        .to_dict()
    )

    # STRATEGY 3: DAY-OF-WEEK OPTIMIZATION
    # One quantity for every bread and weekday.
    by_day = (
        data.groupby(["bread", "weekday"])["assumed_demand"]
        .apply(lambda x: minimum_order(x, service))
        .to_dict()
    )

    # STRATEGY 1: CURRENT ORDERS
    data["current_Q"] = data["current_order"]
```

```
154
155     # STRATEGY 2: FIXED OPTIMAL ORDERS
156     data["fixed_Q"] = data["bread"].map(fixed)
157
158     # STRATEGY 3: WEEKDAY OPTIMAL ORDERS
159     data["weekday_Q"] = [
160         by_day[(bread, weekday)]
161         for bread, weekday in zip(data["bread"], data["weekday"])
162     ]
163
164     # ------------------------------------------------
165     # 5. EVALUATE EACH STRATEGY
166     # ------------------------------------------------
167
168     # This function evaluates one ordering strategy. It returns its waste,
169     # waste cost, service level, and unmet demand.
170     def metrics(data, quantity_column):
171         Q = data[quantity_column]
172         D = data["assumed_demand"]
173
174         # Waste = max(Q - D, 0)
175         waste = np.maximum(Q - D, 0)
176
177         # Unmet demand = max(D - Q, 0)
178         unmet = np.maximum(D - Q, 0)
179
180         return {
181             "wasted_units": int(waste.sum()),
182             "waste_cost": float(
183                 (waste * data["wholesale_cost"]).sum()
184             ),
185             # Share of all bread-day observations covered by order:
186             "service_level_pct": float(
187                 np.mean(D <= Q) * 100
188             ),
189             "unmet_units": int(unmet.sum())
190         }
191
192     results = pd.DataFrame({
193         "Current": metrics(data, "current_Q"),
194         "Fixed optimized": metrics(data, "fixed_Q"),
195         "Weekday optimized": metrics(data, "weekday_Q")
196     }).T
197
198     return fixed, by_day, results
199
200
201 # ------------------------------------------------
202 # 6. RUN THE MODEL
203 # ------------------------------------------------
204
205 if __name__ == "__main__":
206     # First: No additional demand on sold-out days
207     fixed, weekday, results = make_policies(sellout_extra=0)
208
209     print("FIXED OPTIMAL QUANTITIES:")
210     print(fixed)
211
212     print("\nWEEKDAY OPTIMAL QUANTITIES:")
```

```
213
214     schedule = pd.Series(weekday).unstack(0)
215     schedule = schedule.reindex([
216         "Monday", "Tuesday", "Wednesday", "Thursday",
217         "Friday", "Saturday", "Sunday"
218     ])
219     print(schedule)
220
221     print("\nSTRATEGY COMPARISON:")
222     print(results.round(2))
223
224     # ------------------------------------------------
225     # 7. SENSITIVITY ANALYSIS
226     # ------------------------------------------------
227
228     for extra in [0, 1, 2]:
229         fixed, weekday, results = make_policies(
230             sellout_extra=extra
231         )
232         print(f"\nADDITIONAL DEMAND ON SELLOUTS: {extra}")
233         print(results.round(2))
234
235
236 # This function resamples historical demand and tests all three strategies.
237 # It returns simulated performance and paired waste comparisons.
238 def monte_carlo(simulations=10000, seed=381, sellout_extra=0):
239     fixed, by_day, _ = make_policies(sellout_extra=sellout_extra)
240     data = df.copy()
241     data["assumed_demand"] = (
242         data["estimated_sales"]
243         + np.where(data["waste"] == 0, sellout_extra, 0)
244     )
245
246     rng = np.random.default_rng(seed)
247     simulated_demand = np.empty((simulations, len(data)), dtype=int)
248
249     for column, row in data.reset_index(drop=True).iterrows():
250         demand_pool = data.loc[
251             (data["bread"] == row["bread"])
252             & (data["weekday"] == row["weekday"]),
253             "assumed_demand"
254         ].to_numpy()
255         simulated_demand[:, column] = rng.choice(
256             demand_pool, size=simulations, replace=True
257         )
258
259     quantities = {
260         "Current": data["current_order"].to_numpy(),
261         "Fixed optimized": data["bread"].map(fixed).to_numpy(),
262         "Weekday optimized": np.array([
263             by_day[(bread, weekday)]
264             for bread, weekday in zip(data["bread"], data["weekday"])
265         ])
266     }
267     costs = data["wholesale_cost"].to_numpy()
268     simulated = {}
269
270     for strategy, quantity in quantities.items():
271         waste = np.maximum(quantity - simulated_demand, 0)
```

```
        unmet = np.maximum(simulated_demand - quantity, 0)
        simulated[strategy] = {
            "waste": waste.sum(axis=1),
            "cost": (waste * costs).sum(axis=1),
            "unmet": unmet.sum(axis=1),
            "service": (simulated_demand <= quantity).mean(axis=1) * 100
        }

    summary_rows = []
    for strategy, values in simulated.items():
        summary_rows.append({
            "strategy": strategy,
            "mean_waste": values["waste"].mean(),
            "waste_95_low": np.quantile(values["waste"], 0.025),
            "waste_95_high": np.quantile(values["waste"], 0.975),
            "mean_waste_cost": values["cost"].mean(),
            "mean_service_pct": values["service"].mean(),
            "mean_unmet": values["unmet"].mean()
        })

    comparison_rows = []
    current_waste = simulated["Current"]["waste"]
    for strategy in ["Fixed optimized", "Weekday optimized"]:
        reduction = current_waste - simulated[strategy]["waste"]
        comparison_rows.append({
            "strategy": strategy,
            "mean_waste_reduction": reduction.mean(),
            "reduction_95_low": np.quantile(reduction, 0.025),
            "reduction_95_high": np.quantile(reduction, 0.975),
            "probability_less_waste": np.mean(reduction > 0)
        })

    return (
        pd.DataFrame(summary_rows).set_index("strategy"),
        pd.DataFrame(comparison_rows).set_index("strategy")
    )


if __name__ == "__main__":
    simulation_summary, simulation_tests = monte_carlo()
    print("\nMONTE CARLO RESULTS (10,000 RUNS):")
    print(simulation_summary.round(2).to_string())
    print("\nPAIRED WASTE COMPARISONS:")
    print(simulation_tests.round(3).to_string())
```